

Pet Territory Wars — Engine Architecture v1.0

Doküman sürümü: 1.0.0

Motor dili: Go

Mobil istemci: Flutter

Kapsam: Faz 0 Backend Simulation + Faz 1 Barebone Mobile

Mimari yaklaşım: Modüler monolith içinde bağımsız business engine

1. Amaç

Bu doküman, Pet Territory Wars oyun motorunun mimari sınırlarını ve çalışma biçimini tanımlar.

Motorun temel amacı:

Dış dünyadan aldığı yürüyüş, oyuncu, bölge ve kural verilerini değerlendirerek deterministik oyun sonuçları üretmek.

Motor:

- İş kurallarını bilir.
- Hesaplama yapar.
- Sonuç üretir.
- Domain event üretir.
- Veritabanına yazmaz.
- API çağırısı yapmaz.
- Push notification göndermez.
- Kullanıcı arayüzünü bilmez.
- Flutter kodunu bilmez.

Motorun davranışı şu prensibe dayanır:

Same Input + Same State + Same RuleSet
=
Same Output

2. Ana Mimari Karar

MVP'de birden fazla bağımsız motor veya mikroservis kurulmayacaktır.

Tek bir oyun motoru olacaktır:

Game Engine

Bu motorun içinde küçük ve belirli sorumluluklara sahip bileşenler bulunacaktır:

Game Engine
├ Validation
├ Movement
├ Territory
├ Scoring
└ Event Builder

Bunlar ayrı uygulamalar veya ayrı deploy edilen servisler değildir.

Hepsi aynı Go modülü içinde çalışır.

Ana giriş noktası:

```
result := engine.ResolveWalk(ctx, input, rules)
```

3. Mimari Prensipler

3.1 Tek Sorumluluk

Her bileşen yalnızca kendi alanındaki hesabı yapar.

Örnek:

- Validation yürüyüşün geçerliliğini belirler.
- Movement rotayı işler.
- Territory bölge sonuçlarını hesaplar.
- Scoring skor değişimlerini hesaplar.
- Event Builder domain event üretir.

Validation bileşeni skor hesaplamaz.

Territory bileşeni veritabanına yazmaz.

Event Builder bildirim göndermez.

3.2 Determinizm

Motor hesaplama sırasında dış kaynaklardan veri çekmez.

Motor içinde aşağıdakiler kullanılmaz:

- `time.Now()`
- Rastgele sayı üretimi
- Veritabanı sorgusu
- HTTP isteği
- Redis sorgusu
- Global mutable state
- Cihaz bilgisine doğrudan erişim

Gerekli bütün veriler motora input olarak verilir.

Yanlış:

```
func ResolveWalk(input ResolveWalkInput) Result {  
    now := time.Now()  
    hex := database.FindHex(...)  
    rules := redis.GetRules(...)  
}
```

Doğru:

```
func ResolveWalk(  
    input ResolveWalkInput,  
    rules RuleSet,  
) ResolveWalkResult {  
    // Tüm hesaplama yalnızca input ve rules üzerinden yapılır.  
}
```

3.3 Saf Hesaplama

Motor mümkün olduğunca saf fonksiyonlardan oluşmalıdır.

Örnek:

```
func ApplyDecay(  
    current int,  
    lastUpdatedAt time.Time,  
    evaluatedAt time.Time,  
    rules DecayRules,  
) int
```

Bu fonksiyon:

- Veritabanı bilmez.

- Oyuncu bilmez.
- HTTP bilmez.
- Sadece dominance hesaplar.

3.4 Versiyonlanmış Kurallar

Her yürüyüş belirli bir RuleSet sürümüyle hesaplanır.

```
RuleSetVersion = "mvp-0.1.0"
```

Sonradan kurallar değiştiğinde eski yürüyüşlerin neden farklı sonuç verdiği açıklanabilir.

Her motor sonucu şunları içermelidir:

```
Engine Version
RuleSet Version
Input Hash
Evaluation Time
```

3.5 MVP Odaklı Tasarım

İlk sürümde şu yapılar kurulmayacaktır:

- Plugin sistemi
- Script tabanlı rule engine
- Karmaşık dependency injection framework
- Her kural için ayrı interface
- Mikroservis mimarisi
- Event sourcing altyapısının tam sürümü
- Dağıtık transaction yapısı
- Generic workflow engine

Yeni soyutlama yalnızca gerçek ihtiyaç ortaya çıktığında eklenecektir.

4. Katmanlar

Genel backend yapısı şu katmanlardan oluşacaktır:

```
Transport Layer
↓
Application Layer
↓
```

```
Game Engine
  ↓
Repository Interfaces
  ↓
Infrastructure Layer
```

Ancak Game Engine repository kullanmaz.

Doğru ilişki:

```
Application Layer
├─ Repository'den gerekli state'i alır
├─ RuleSet'i alır
├─ Engine'i çağırır
└─ Engine sonucunu kaydeder
```

Motor:

```
Input alır
Result döndürür
```

5. Önerilen Proje Yapısı

```
pet-territory-backend/
├─ cmd/
│   ├── api/
│   │   └─ main.go
│   └─ simulator/
│       └─ main.go
├─ internal/
│   ├── application/
│   │   ├── walk/
│   │   │   ├── resolve_walk.go
│   │   │   └─ result.go
│   │   └─ season/
│   │       └─ reset_season.go
│   ├── gameengine/
│   │   ├── engine.go
│   │   ├── input.go
│   │   ├── result.go
│   │   ├── rules.go
│   │   └─ validation.go
```

```
|
|
|   |— movement.go
|   |— territory.go
|   |— scoring.go
|   |— events.go
|   |— types.go
|   |— errors.go
|
|   |— domain/
|       |— walk.go
|       |— territory.go
|       |— player.go
|       |— season.go
|
|   |— repository/
|       |— walk_repository.go
|       |— territory_repository.go
|       |— player_repository.go
|       |— rules_repository.go
|
|   |— infrastructure/
|       |— postgres/
|       |— redis/
|       |— h3adapter/
|       |— clock/
|
|   |— transport/
|       |— http/
|
|— migrations/
|— configs/
|— testdata/
|— go.mod
```

MVP’de bütün bu klasörlerin ilk günden dolu olması gerekmez.

Minimum başlangıç:

```
cmd/
internal/application/
internal/gameengine/
internal/domain/
internal/infrastructure/
```

6. Game Engine Paket Yapısı

6.1 engine.go

Motorun tek public giriş noktasını içerir.

```
type Engine struct {  
    movementResolver MovementResolver  
}
```

MVP'de Engine yapısı mümkün olduğunca küçük tutulmalıdır.

```
func (e Engine) ResolveWalk(  
    input ResolveWalkInput,  
    rules RuleSet,  
) ResolveWalkResult
```

Engine'in görevi hesaplama sırasını yönetmektir.

Business detaylarını doğrudan kendi içinde yazmamalıdır.

6.2 input.go

Motorun ihtiyaç duyduğu bütün giriş modellerini içerir.

```
type ResolveWalkInput struct {  
    WalkID           WalkID  
    PlayerID         PlayerID  
    StartedAt        time.Time  
    EndedAt          time.Time  
    EvaluatedAt      time.Time  
    Route            []LocationPoint  
    ExistingHexes    map[HexID]HexState  
    ActiveCity       CityBoundary  
    EngineVersion    string  
}
```

Motor eksik state'i kendisi aramaz.

Örneğin yürüyüşün dokunduğu hex'lerin mevcut durumları application katmanı tarafından input içine eklenir.

6.3 result.go

Motorun ürettiği sonucu içerir.

```
type ResolveWalkResult struct {
    Status           WalkStatus
    Validation        WalkValidationResult
    Metrics           WalkMetrics
    VisitedHexes     []VisitedHex
    HexChanges        []HexChange
    ScoreChanges      []ScoreChange
    Events            []DomainEvent
    Metadata          CalculationMetadata
}
```

Result içinde persistence modeli bulunmaz.

Örneğin:

```
type HexChange struct {
    HexID           HexID
    PreviousOwnerID *PlayerID
    NewOwnerID      *PlayerID
    PreviousDominance int
    NewDominance     int
    ChangeType       HexChangeType
}
```

Application katmanı bu sonucu veritabanı modellerine dönüştürür.

6.4 rules.go

Bütün değiştirilebilir sayısal parametreleri içerir.

RuleSet düz bir yapı yerine gruplandırılmış olmalıdır.

```
type RuleSet struct {
    Version  string
    Walk     WalkRules
    Movement MovementRules
    Territory TerritoryRules
    Season   SeasonRules
}
```

WalkRules

```
type WalkRules struct {
    MinDurationSeconds int
    MinDistanceMeters  float64
    MinValidRouteRatio float64
    MaxSpeedMPS        float64
    MaxAccuracyMeters  float64
    MaxJumpMeters      float64
}
```

MovementRules

```
type MovementRules struct {
    H3Resolution          int
    MinHexPresenceSeconds int
    MinHexDistanceMeters  float64
}
```

TerritoryRules

```
type TerritoryRules struct {
    MaxDominance          int
    InitialDominance      int
    OwnerVisitGain        int
    EnemyAttackDamage     int
    DailyDecay            int
    MinimumOwnedDominance int
    ThreatThreshold       int
}
```

SeasonRules

```
type SeasonRules struct {
    WeekStartsOn time.Weekday
    ResetHourUTC int
}
```

Bu yapı sayesinde yeni bir değer ilgili gruba eklenir.

Örneğin:

```
type TerritoryRules struct {
    MaxDominance          int
    InitialDominance      int
    OwnerVisitGain        int
}
```

```
OwnerVisitDailyLimit  int
EnemyAttackDamage     int
DailyDecay             int
}
```

Bütün motoru yeniden tasarlamak gerekmez.

7. Engine Bileşenleri

7.1 Validation Component

Dosya:

```
validation.go
```

Görevi:

- Input doğrulamak
- Timestamp sırasını kontrol etmek
- GPS noktalarını değerlendirmek
- Segmentlerin geçerliliğini belirlemek
- Yürüyüşün valid, partially valid veya invalid olduğuna karar vermek

Örnek çıktı:

```
type WalkValidationResult struct {
    Status           WalkValidationStatus
    Reasons          []ValidationReason
    ValidPointCount  int
    InvalidPointCount int
    ValidSegmentCount int
    InvalidSegments  []InvalidSegment
    ValidRouteRatio  float64
}
```

Validation iş kuralları RuleSet'ten okunur.

Yanlış:

```
if speed > 4.5 {
    invalid = true
}
```

Doğru:

```
if speed > rules.Walk.MaxSpeedMPS {  
    invalid = true  
}
```

7.2 Movement Component

Dosya:

```
movement.go
```

Görevi:

- Geçerli GPS segmentlerini almak
- Rota mesafesini hesaplamak
- Şehir sınırı dışındaki segmentleri filtrelemek
- Rotayı gerektiğinde interpolate etmek
- Rotayı H3 hücrelerine çevirmek
- Her hex içindeki süre ve mesafeyi hesaplamak
- Tekrarlanan hex ziyaretlerini birleştirmek

Örnek çıktı:

```
type MovementResult struct {  
    TotalDistanceMeters float64  
    ValidDistanceMeters float64  
    DurationSeconds     int  
    VisitedHexes        []VisitedHex  
}
```

```
type VisitedHex struct {  
    HexID           HexID  
    PresenceSeconds int  
    DistanceMeters  float64  
    EntryCount      int  
    FirstEnteredAt  time.Time  
    LastExitedAt    time.Time  
}
```

Movement bileşeni ownership veya dominance bilmez.

Yalnızca oyuncunun hangi hex'leri ne ölçüde ziyaret ettiğini üretir.

7.3 Territory Component

Dosya:

```
territory.go
```

Görevi:

Her ziyaret edilen hex için mevcut state ile yeni sonucu karşılaştırmak.

İşlem türleri:

```
EMPTY_CAPTURE  
OWNER_DEFENSE  
ENEMY_ATTACK  
OWNERSHIP_TRANSFER  
NO_CHANGE
```

Territory bileşeni şu sırayı izler:

1. Mevcut state'i al
2. Efektif decay hesapla
3. Ziyaretin türünü belirle
4. Capture, defense veya attack uygula
5. Ownership değişimini değerlendir
6. HexChange üret

Örnek:

```
func ResolveHex(  
    playerID PlayerID,  
    visited VisitedHex,  
    current HexState,  
    evaluatedAt time.Time,  
    rules TerritoryRules,  
) HexChange
```

Bu fonksiyon tek bir hex'i çözer.

Bütün rota için:

```
func ResolveTerritories(  
    playerID PlayerID,  
    visited []VisitedHex,  
    current map[HexID]HexState,
```

```
        evaluatedAt time.Time,  
        rules TerritoryRules,  
    ) []HexChange
```

7.4 Scoring Component

Dosya:

```
scoring.go
```

Görevi:

Hex değişikliklerinden skor değişikliklerini üretmek.

Scoring doğrudan hex state değiştirmez.

Girdi:

```
HexChanges  
Current Player Score State
```

Çıktı:

```
type ScoreChange struct {  
    PlayerID PlayerID  
    Metric   ScoreMetric  
    Delta    int  
    Reason   ScoreChangeReason  
}
```

Örnek:

```
EMPTY_CAPTURE  
→ WeeklyActiveHex +1  
→ WeeklyCapture +1  
→ LifetimeCapture +1
```

```
RIVAL_CAPTURE  
→ Attacker WeeklyActiveHex +1  
→ Defender WeeklyActiveHex -1  
→ Attacker WeeklySteal +1
```

MVP'de leaderboard sorgusu motorun içinde yapılmaz.

Motor yalnızca skor delta'ları üretir.

7.5 Event Builder Component

Dosya:

```
events.go
```

Görevi:

Validation, territory ve scoring sonuçlarını domain event'lere çevirmek.

Örnek event:

```
type DomainEvent struct {  
    Type      EventType  
    AggregateID string  
    OccurredAt time.Time  
    Payload    any  
}
```

MVP event türleri:

```
WALK_VALIDATED  
WALK_PARTIALLY_VALIDATED  
WALK_REJECTED  
EMPTY_HEX_CAPTURED  
HEX_DEFENDED  
HEX_ATTACKED  
HEX_UNDER_THREAT  
HEX_CAPTURED_FROM_RIVAL  
HEX_LOST  
PLAYER_SCORE_CHANGED
```

Event Builder:

- Kafka bilmez.
- RabbitMQ bilmez.
- Push notification bilmez.
- Event'i yayınlamaz.

Sadece sonuç listesine event ekler.

8. Engine İşlem Akışı

`ResolveWalk` akışı şu sırayla çalışmalıdır:

1. Input validation
2. GPS point validation
3. Segment validation
4. City boundary filtering
5. Walk metrics calculation
6. Walk status decision
7. Route interpolation
8. H3 conversion
9. Hex visit aggregation
10. Decay calculation
11. Territory resolution
12. Ownership transfer
13. Score calculation
14. Domain event creation
15. Metadata creation
16. Result return

Örnek orchestration:

```
func (e Engine) ResolveWalk(  
    input ResolveWalkInput,  
    rules RuleSet,  
) ResolveWalkResult {  
    validation := ValidateWalk(input, rules.Walk)  
  
    if validation.Status == WalkInvalid {  
        return BuildInvalidWalkResult(  
            input,  
            rules,  
            validation,  
        )  
    }  
  
    movement := e.movementResolver.Resolve(  
        input.Route,  
        input.ActiveCity,  
        rules.Movement,  
    )  
  
    hexChanges := ResolveTerritories(  
        input.PlayerID,  
        movement.VisitedHexes,  
        input.ExistingHexes,  
        input.EvaluatedAt,
```

```

        rules.Territory,
    )

    scoreChanges := CalculateScoreChanges(
        input.PlayerID,
        hexChanges,
    )

    events := BuildDomainEvents(
        input,
        validation,
        hexChanges,
        scoreChanges,
    )

    return ResolveWalkResult{
        Status:      WalkResolved,
        Validation:   validation,
        Metrics:      movement.Metrics(),
        VisitedHexes: movement.VisitedHexes,
        HexChanges:   hexChanges,
        ScoreChanges: scoreChanges,
        Events:       events,
        Metadata:     BuildMetadata(input, rules),
    }
}

```

Bu örnek nihai kod değildir. Bileşenlerin sınırlarını gösterir.

9. Harici Bağımlılıkların Yönetimi

9.1 H3 Bağımlılığı

H3, motorun ihtiyaç duyduğu teknik bir araçtır.

Motorun tamamı doğrudan üçüncü taraf H3 package'ına bağımlı olmamalıdır.

H3 işlemleri küçük bir adapter arkasında tutulabilir:

```

type HexResolver interface {
    CellFromLatLng(
        lat float64,
        lng float64,
        resolution int,
    ) (HexID, error)

    CellsForSegment(

```

```
    from LocationPoint,  
    to LocationPoint,  
    resolution int,  
    ) ([HexID, error)  
}
```

Engine yalnızca bu interface'i kullanır.

Sebebi:

- H3 kütüphanesi değişebilir.
- Testlerde fake resolver kullanılabilir.
- Simülasyonda kontrollü hex sonuçları üretilebilir.
- Third-party tipleri domain içine sızmaz.

MVP'de gerekli görülen temel interface budur.

Her bileşen için interface oluşturmak gerekli değildir.

9.2 Clock

Motor `time.Now()` kullanmayacaktır.

Değerlendirme zamanı input içinde olacaktır:

```
EvaluatedAt time.Time
```

Application katmanı zamanı sağlar.

Bu sayede test ve replay deterministik olur.

9.3 Randomness

MVP motorunda randomness bulunmayacaktır.

Gelecekte NPC veya event sistemi random değer gerektirirse seed dışarıdan verilir:

```
RandomSeed int64
```

Ancak MVP'de buna ihtiyaç yoktur.

10. Error Model

Motor hataları iki gruba ayrılmalıdır.

10.1 Expected Business Results

Bunlar sistem hatası değildir.

Örnek:

```
TOO_SHORT_DISTANCE
TOO_SHORT_DURATION
IMPOSSIBLE_SPEED
OUTSIDE_ACTIVE_CITY
MOCK_LOCATION_DETECTED
```

Bunlar normal result içinde döner:

```
ResolveWalkResult{
    Status: WalkRejected,
    Validation: ...
}
```

Go `error` olarak dönmemelidir.

10.2 Technical Errors

Bunlar hesaplamamanın tamamlanamadığı durumlardır.

Örnek:

```
Invalid RuleSet
Unsupported H3 resolution
Malformed city polygon
Hex resolver failure
Internal invariant violation
```

Bu durumda:

```
func (e Engine) ResolveWalk(
    input ResolveWalkInput,
```

```
rules RuleSet,  
) (ResolveWalkResult, error)
```

kullanılabilir.

Örnek typed error:

```
type EngineError struct {  
    Code    EngineErrorCode  
    Message string  
    Cause   error  
}
```

Kullanıcıya gösterilecek metni motor üretmez.

11. RuleSet Validation

RuleSet motor çağrısından önce doğrulanmalıdır.

Örnek:

```
func (r RuleSet) Validate() error
```

Kontroller:

```
Version boş olamaz.  
H3 resolution desteklenen aralıkta olmalı.  
MaxDominance > 0 olmalı.  
InitialDominance <= MaxDominance olmalı.  
OwnerVisitGain >= 0 olmalı.  
EnemyAttackDamage > 0 olmalı.  
DailyDecay >= 0 olmalı.  
MinDistance ve MinDuration negatif olamaz.  
ThreatThreshold <= MaxDominance olmalı.
```

Geçersiz RuleSet ile hesaplama yapılmamalıdır.

RuleSet validation her hex için tekrar çalıştırılmaz.

Application başlatılırken veya rules yüklenirken kontrol edilir.

12. State Yönetimi

Motor state tutmaz.

Yanlış:

```
type Engine struct {  
    currentHexes map[HexID]HexState  
    playerScores map[PlayerID]int  
}
```

Doğru:

```
type Engine struct {  
    hexResolver HexResolver  
}
```

Gerekli oyun state'i her çağrıda input ile verilir.

Bu sayede motor:

- Stateless olur.
- Paralel çalışabilir.
- Kolay test edilir.
- Yatay ölçeklenebilir.
- Simulation CLI içinde kullanılabilir.
- API içinde kullanılabilir.

13. Application Layer ile İlişki

Application katmanı kullanım senaryosunu yönetir.

Örnek:

```
type ResolveWalkUseCase struct {  
    walkRepository      WalkRepository  
    territoryRepository TerritoryRepository  
    ruleRepository      RuleRepository  
    engine              gameengine.Engine  
}
```

Akış:

1. Walk verisini repository'den al
2. Aktif RuleSet'i al
3. Rotanın dokunacağı hex'leri belirle
4. Mevcut hex state'lerini repository'den al
5. ResolveWalkInput oluştur
6. Engine'i çağır
7. Transaction başlat
8. HexChanges kaydet
9. ScoreChanges kaydet
10. Walk sonucunu kaydet
11. Domain event outbox kayıtlarını oluştur
12. Transaction tamamla

Motor transaction bilmez.

Motor repository bilmez.

Motor event outbox bilmez.

14. Database ile İlişki

Engine database modelini kullanmamalıdır.

Yanlış:

```
func ResolveWalk(  
    walk postgres.WalkModel,  
    hexes []postgres.HexModel,  
)
```

Doğru:

```
func ResolveWalk(  
    input ResolveWalkInput,  
    rules RuleSet,  
)
```

Application katmanında mapper kullanılır:

```
func HexModelToState(model HexModel) HexState
```

```
func HexChangeToPersistence(change HexChange) HexUpdate
```

Bu ayırım sayesinde database şeması değiştiğinde engine etkilenmez.

15. API ile İlişki

API katmanı engine'i doğrudan çağırmamalıdır.

Doğru akış:

```
HTTP Handler
  ↓
ResolveWalk Use Case
  ↓
Game Engine
```

Handler'ın görevi:

- Request parse etmek
- Authentication bilgisi almak
- Input DTO doğrulamak
- Use case çağırmak
- Response DTO üretmek

Handler iş kuralı uygulamaz.

Yanlış:

```
if request.Distance < 300 {
    return badRequest
}
```

Doğru:

```
result, err := resolveWalkUseCase.Execute(...)
```

Minimum mesafe business rule'dur ve engine tarafından değerlendirilir.

16. Flutter ile İlişki

Flutter uygulaması motor mantığını yeniden yazmamalıdır.

Flutter'ın görevi:

- GPS noktalarını toplamak

- Walk başlatmak ve bitirmek
- API'ye rota göndermek
- Motor sonucunu göstermek
- Haritada hex değişikliklerini çizmek
- Kullanıcıya durum mesajı göstermek

Flutter içinde şu değerler authoritative olmamalıdır:

```
Attack damage
Dominance gain
Decay
Ownership transfer
Valid walk kararı
Weekly score
```

Flutter tahmini UI gösterebilir ancak gerçek sonuç backend motorundan gelir.

17. Domain Event Yaklaşımı

Motor event üretir ancak event publish etmez.

Örnek çıktı:

```
[]DomainEvent{
  {
    Type: HexAttacked,
    AggregateID: "hex_abc",
    OccurredAt: input.EvaluatedAt,
  },
  {
    Type: PlayerScoreChanged,
    AggregateID: "player_xyz",
    OccurredAt: input.EvaluatedAt,
  },
}
```

Application katmanı bunları:

- Outbox tablosuna yazabilir.
- Analytics'e gönderebilir.
- Push notification işlemine dönüştürebilir.
- Audit log'a kaydedebilir.

Motor bunlardan hangisinin yapılacağını bilmez.

18. Test Stratejisi

18.1 Unit Tests

Her saf hesaplama fonksiyonu bağımsız test edilir.

Örnek:

```
ApplyDecay  
ValidateSegment  
CalculateSpeed  
ResolveEmptyHex  
ResolveOwnerVisit  
ResolveEnemyAttack  
ResolveOwnershipTransfer  
CalculateScoreChanges
```

Örnek test adı:

```
func TestApplyDecay_StopsAtMinimumOwnedDominance(t *testing.T)
```

18.2 Table-Driven Tests

Go table-driven test yapısı kullanılmalıdır.

```
func TestResolveEnemyAttack(t *testing.T) {  
    tests := []struct {  
        name      string  
        current   int  
        damage    int  
        expected  int  
    }{  
        {  
            name:      "reduces dominance",  
            current:  100,  
            damage:   15,  
            expected: 85,  
        },  
        {  
            name:      "does not go below zero",  
            current:  10,  
            damage:   15,  
            expected: 0,  
        },  
    }  
}
```

```
// ...  
}
```

18.3 Golden Tests

Tam yürüyüş senaryoları JSON input ve output dosyalarıyla test edilebilir.

```
testdata/  
├─ walk_empty_capture_input.json  
├─ walk_empty_capture_output.json  
├─ walk_enemy_attack_input.json  
├─ walk_enemy_attack_output.json  
├─ walk_partial_invalid_input.json  
└─ walk_partial_invalid_output.json
```

Rule değiştiğinde output farkı açıkça görülebilir.

18.4 Replay Tests

Aynı input yüzlerce kez çalıştırıldığında aynı sonuç alınmalıdır.

Kontrol:

```
HexChanges aynı mı?  
ScoreChanges aynı mı?  
Event sırası aynı mı?  
InputHash aynı mı?
```

Map iteration sırası deterministik olmadığı için map sonuçları doğrudan event sırası üretmekte kullanılmamalıdır.

Hex ID'ler işlenmeden önce sıralanmalıdır.

```
sort.Slice(hexIDs, ...)
```

18.5 Simulation Tests

Aynı engine simulator CLI tarafından kullanılacaktır.

```
go run ./cmd/simulator
--players=100
--days=30
--rules=mvp-0.1.0
```

Simülasyon:

- Gerçek engine'i kullanır.
- Ayrı bir simülasyon business logic'i yazmaz.
- Yapay rota ve state üretir.
- Sonuçları raporlar.

Ölçümler:

```
Walk başına hex sayısı
Ownership değişim sayısı
Ortalama dominance
Hex yaşam süresi
Oyuncu başına aktif hex
Harita kilitlenme oranı
Yeni oyuncu ilerleme oranı
```

19. Concurrency ve Transaction

Motor stateless olduğu için aynı anda birden fazla yürüyüş hesaplanabilir.

Ancak aynı hex'e aynı anda iki yürüyüş saldırabilir.

Bu problem engine problemi değil, application ve database concurrency problemidir.

Örnek:

```
Player A → Hex X saldırısı
Player B → Hex X saldırısı
```

İki motor çağrısı da aynı eski state'i kullanırsa conflict oluşabilir.

Çözüm ileride database tasarımında ele alınacaktır:

- Optimistic locking
- Version field
- Row lock
- Retry
- Serializable transaction gerekip gerekmediği

Motor her HexChange içinde beklediği önceki state'i taşımalıdır:

```
type HexChange struct {
    HexID           HexID
    ExpectedVersion int64
    PreviousOwnerID *PlayerID
    PreviousDominance int
    NewOwnerID      *PlayerID
    NewDominance    int
}
```

Repository update sırasında version uyuşmazsa application işlemi tekrar hesaplayabilir.

20. Observability

Motor doğrudan log yazmak zorunda değildir.

Tercih edilen yaklaşım:

- Motor structured result üretir.
- Application log ve metric üretir.

Result metadata:

```
type CalculationMetadata struct {
    EngineVersion string
    RuleSetVersion string
    InputHash      string
    EvaluatedAt    time.Time
    DurationMicros int64
}
```

`DurationMicros` deterministik sonuç parçası değildir. Teknik gözlem amacıyla wrapper tarafından eklenebilir.

Motor sonucu üzerinden ölçülebilecek metrikler:

```
walk_validation_failed_total
walk_partially_valid_total
hex_capture_total
hex_attack_total
hex_transfer_total
engine_calculation_duration
engine_error_total
```

21. Güvenlik Sınırı

Motor anti-cheat kararlarını değerlendirebilir ancak cihazı kendisi incelemeyebilir.

Input içindeki güvenlik sinyalleri:

```
type LocationPoint struct {  
    Latitude      float64  
    Longitude     float64  
    RecordedAt    time.Time  
    AccuracyMeters float64  
    IsMockLocation bool  
}
```

Gelecekte eklenebilir:

```
DeviceIntegrityScore  
GPSPProvider  
SensorConsistencyScore  
BackgroundExecutionState
```

Bu sinyalleri mobil ve application katmanı toplar.

Motor bunları RuleSet'e göre değerlendirir.

Motor:

- Cihaz API'si çağırılmaz.
- Google Play Integrity çağırılmaz.
- Apple App Attest çağırılmaz.
- IP adresi kontrol etmez.

22. SOLID Uygulaması

Single Responsibility Principle

Dosya ve fonksiyonlar belirli sorumluluğa sahiptir.

```
validation.go → geçerlilik  
movement.go   → rota ve hex  
territory.go  → dominance ve ownership  
scoring.go    → skor delta  
events.go     → domain event
```

Open/Closed Principle

Her ihtimal için baştan abstraction kurulmaz.

Ancak RuleSet değerleri değiştirilerek motor davranışı değiştirilebilir.

Yeni algoritma gerçekten gerektiğinde strategy eklenebilir.

Örnek:

Bugün:

```
ApplyLinearDecay(...)
```

Gelecekte iki gerçek decay algoritması gerektiğinde:

```
type DecayCalculator interface {  
    Apply(input DecayInput) DecayResult  
}
```

Interface ihtiyaçtan önce oluşturulmaz.

Liskov Substitution Principle

MVP'de az sayıda interface olduğu için risk sınırlıdır.

`HexResolver` implementasyonları aynı sözleşmeye uymalıdır.

Fake resolver ile gerçek H3 resolver farklı sonuç formatı üretmemelidir.

Interface Segregation Principle

Büyük bir generic interface yerine küçük interface kullanılır.

Yanlış:

```
type GeoService interface {  
    ResolveHex(...)  
    ReverseGeocode(...)  
    FindCity(...)  
    DrawPolygon(...)
```

```
    CalculateRoute(...)  
}
```

Doğru:

```
type HexResolver interface {  
    CellFromLatLng(...)  
    CellsForSegment(...)  
}
```

Dependency Inversion Principle

Engine üçüncü taraf H3 package'ına değil kendi küçük abstraction'ına bağımlıdır.

Application, somut PostgreSQL repository yerine repository sözleşmelerine bağımlı olabilir.

Ancak engine repository'ye hiç bağımlı değildir.

23. İlk Sürümde Public ve Private API

`gameengine` paketinin dışarı açacağı minimum public API:

```
type Engine  
type RuleSet  
type ResolveWalkInput  
type ResolveWalkResult  
type EngineError  
  
func NewEngine(hexResolver HexResolver) Engine  
func (e Engine) ResolveWalk(  
    input ResolveWalkInput,  
    rules RuleSet,  
) (ResolveWalkResult, error)
```

Diğer hesaplama fonksiyonları mümkün olduğunca package-private kalmalıdır.

```
func applyDecay(...)  
func validateSegment(...)  
func resolveHex(...)  
func calculateScoreChanges(...)
```

Bu yaklaşım dış katmanların engine detaylarına bağımlı olmasını engeller.

24. Engine Oluşturma

Örnek:

```
hexResolver := h3adapter.NewResolver()

gameEngine := gameengine.NewEngine(hexResolver)
```

Engine constructor:

```
func NewEngine(hexResolver HexResolver) Engine {
    if hexResolver == nil {
        panic("hex resolver is required")
    }

    return Engine{
        hexResolver: hexResolver,
    }
}
```

Motor içine repository, logger, Redis veya HTTP client verilmez.

25. Engine Versioning

Engine kod sürümü ve RuleSet sürümü farklı kavramlardır.

```
EngineVersion = "1.0.0"
RuleSetVersion = "mvp-0.1.0"
```

Engine değişmeden RuleSet değişebilir:

```
Engine 1.0.0 + Rules mvp-0.1.0
Engine 1.0.0 + Rules mvp-0.2.0
```

Algoritma değiştiğinde EngineVersion değişir:

```
Engine 1.1.0 + Rules mvp-0.2.0
```

Semantik yaklaşım:

MAJOR → Geriye dönük uyumsuz hesaplama değişikliği
MINOR → Yeni hesaplama davranışı
PATCH → Sonucu değiştirmeyen hata düzeltmesi

Sonucu değiştiren bir bug fix varsa patch yerine yeni rule veya engine sürümü düşünülmelidir.

26. RuleSet Yükleme

Motor RuleSet'in nereden geldiğini bilmez.

RuleSet:

- Go config dosyasından
- JSON'dan
- PostgreSQL'den
- Admin panelinden
- Environment bazlı config'ten

yüklenebilir.

Application katmanı RuleSet'i hazırlar:

```
rules, err := ruleRepository.GetActive(ctx, cityID)
```

Sonra engine'e verir:

```
result, err := engine.ResolveWalk(input, rules)
```

MVP'de RuleSet'in şehir bazlı olması zorunlu değildir.

İlk sürüm:

```
Global Active RuleSet
```

yeterlidir.

27. Değişiklik Senaryoları

Senaryo 1 — Attack Damage Değişti

Eski:

```
EnemyAttackDamage = 15
```

Yeni:

```
EnemyAttackDamage = 12
```

Sadece yeni RuleSet yayımlanır.

Engine kodu değişmez.

Senaryo 2 — Daily Decay Değişti

Eski:

```
DailyDecay = 5
```

Yeni:

```
DailyDecay = 8
```

Sadece RuleSet değişir.

Senaryo 3 — Linear Decay Yerine Başka Algoritma

Bu yalnızca gerçekten ihtiyaç olursa kod değişikliği gerektirir.

İlk aşamada yeni strategy abstraction eklenmez.

Senaryo 4 — Yeni Walk Validation Kuralı

Örneğin:

```
Minimum valid GPS point count
```

`WalkRules` içine değer eklenir.

Validation bileşeni güncellenir.

Territory ve scoring etkilenmez.

Senaryo 5 — Loop Bonus Eklendi

Gelecekte:

- Movement loop tespit sonucu üretir.
- Territory bu bonusu uygular.
- RuleSet'e `LoopAttackBonus` eklenir.

Mevcut engine akışı korunur.

28. Architecture Decision Records

Önemli mimari kararlar kısa ADR dosyalarında saklanmalıdır.

```
docs/adr/  
├─ 0001-use-go-for-game-engine.md  
├─ 0002-engine-is-stateless.md  
├─ 0003-engine-does-not-access-database.md  
├─ 0004-use-versioned-rule-sets.md  
├─ 0005-use-h3-behind-adapter.md  
└─ 0006-start-as-modular-monolith.md
```

ADR formatı:

```
Title  
Status  
Context  
Decision  
Consequences
```

Bu zorunlu bir bürokrasi değildir.

Yalnızca ileride neden bu kararların alındığını hatırlamak için kullanılır.

29. MVP Mimari Sınırları

MVP için engine aşağıdaki alanları kapsar:

```
Walk validation  
Segment validation  
City boundary filtering  
Route-to-H3 conversion  
Hex visit aggregation
```

- Dominance decay
- Empty capture
- Owner defense
- Enemy attack
- Ownership transfer
- Score delta
- Domain event generation
- Replay metadata

MVP engine aşağıdaki alanları kapsamaz:

- Guild
- Breed bonuses
- Professional walker mode
- Coins
- XP
- Premium boosters
- Sponsored areas
- NPC battles
- Push notification
- Leaderboard query
- Payment
- Authentication
- Map rendering

30. Definition of Done

Engine Architecture v1.0 aşağıdaki koşullar sağlandığında uygulanmış kabul edilir:

1. Engine tek public ResolveWalk girişine sahip.
2. Engine hiçbir repository kullanmıyor.
3. Engine time.Now çağırıyor.
4. Engine HTTP veya Redis çağırısı yapmıyor.
5. RuleSet versiyonlu.
6. H3 bağımlılığı küçük bir adapter arkasında.
7. Validation, movement, territory, scoring ve events ayrılmış.
8. Aynı input aynı sonucu üretiyor.
9. Bütün HexChange sonuçları açıklanabilir.
10. Unit ve golden testler mevcut.
11. Simulator gerçek engine'i kullanıyor.
12. Database modelleri engine içine sızıyor.
13. Flutter business rule hesaplamıyor.
14. Application katmanı persistence ve transaction yönetiyor.
15. Engine sonucu domain event listesi içeriyor.

31. Nihai Mimari Özeti

Pet Territory Wars MVP motoru:

Tek motor
Tek giriş noktası
Birden fazla küçük bileşen
Stateless çalışma
Versiyonlu RuleSet
Deterministik sonuç
Database bağımsızlığı
Flutter bağımsızlığı
Test ve simülasyon uyumluluğu

Ana akış:

Flutter
↓
Go API
↓
Application Use Case
↓
Game Engine
↓
ResolveWalkResult
↓
Application Transaction
↓
Database + Outbox

Motorun temel sorumluluğu:

Verilen yürüyüş ve mevcut oyun durumu sonucunda ne olması gerektiğini hesaplamak.

Application katmanının sorumluluğu:

Motorun ürettiği sonucu güvenli biçimde uygulamak.

Bu ayrım korunduğu sürece oyun kuralları mobil uygulamadan, API'den ve veritabanından bağımsız biçimde geliştirilebilir ve simüle edilebilir.